Project Report
on

REALTIME SPEECH TRANSLATION
at
U. V. Patel College of Engineering

Ganpat University
॥ विद्यया समाजोत्कर्षः ॥

Project Guide :
Prof. Hitesh Patel

Prepared By:
Mr. Bhargav (22012531001)
Mr. Prudhvi (22012531035)
Mr. Chandra (22012531056)
Mr. Murtuja (22012531062)

B. Tech Semester VI
(Computer Engineering/ Artificial Intelligence)

January-June 2025

Submitted to,
Department of Computer Engineering/Information Technology/Computer Engineering-Artificial Intelligence
U.V. Patel College of Engineering
Ganpat University, Ganpat Vidyanagar, Mehsana - 384 012

U.V. PATEL COLLEGE OF ENGINEERING

17/05/2025

C E R T I F I C A T E

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Mr. Bhargav Akshit Kumar Alladi student of **B.Tech. Semester VI (Computer Engineering-Artificial Intelligence)** has completed his full semester Capstone Project-II titled "**Real Time Speech Translation"** satisfactorily in partial fulfillment of the requirement of Bachelor of Technology degree of Computer Engineering-Artificial Intelligence at Ganpat University, Ganpat Vidyanagar, Mehsana in the year 2024-2025.

Project Guide

Sign

Prof. Hitesh Patel

Dr. Paresh M. Solanki
Head, Computer Engineering

U.V. PATEL COLLEGE OF ENGINEERING

17/05/2025

C E R T I F I C A T E

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Mr. Prudhvi Kumar Mallireddy student of **B.Tech. Semester VI (Computer Engineering-Artificial Intelligence)** has completed his full semester Capstone Project-II titled **"Real Time Speech Translation"** satisfactorily in partial fulfillment of the requirement of Bachelor of Technology degree of Computer Engineering-Artificial Intelligence at Ganpat University, Ganpat Vidyanagar, Mehsana in the year 2024-2025.

Project Guide

Sign

Prof. Hitesh Patel

Dr. Paresh M. Solanki
Head, Computer Engineering

U.V. PATEL COLLEGE OF ENGINEERING

17/05/2025

C E R T I F I C A T E

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Mr. Chandrasekhar Vadapalli student of **B.Tech. Semester VI (Computer Engineering-Artificial Intelligence)** has completed his full semester Capstone Project-II titled **"Real Time Speech Translation"** satisfactorily in partial fulfillment of the requirement of Bachelor of Technology degree of Computer Engineering-Artificial Intelligence at Ganpat University, Ganpat Vidyanagar, Mehsana in the year 2024-2025.

Project Guide

Sign

Prof. Hitesh Patel

Dr. Paresh M. Solanki
Head, Computer Engineering

U.V. PATEL COLLEGE OF ENGINEERING

17/05/2025

C E R T I F I C A T E

TO WHOMSOEVER IT MAY CONCERN

This is to certify that Mr. Shaik Murtuja Vali student of **B.Tech. Semester VI (Computer Engineering-Artificial Intelligence)** has completed his full semester Capstone Project-II titled **"Real Time Speech Translation"** satisfactorily in partial fulfillment of the requirement of Bachelor of Technology degree of Computer Engineering-Artificial Intelligence at Ganpat University, Ganpat Vidyanagar, Mehsana in the year 2024-2025.

Project Guide

Sign

Prof. Hitesh Patel

Dr. Paresh M. Solanki
Head, Computer Engineering

ACKNOWLEDGMENT

We would like to express our sincere gratitude to all those who contributed to the successful completion of this research on student performance prediction. First and foremost, we extend our heartfelt thanks to our academic supervisor, Prof. Hitesh Patel, for their invaluable guidance, constructive feedback, and continuous support throughout the project. Their expertise and encouragement have been instrumental in shaping the direction and depth of our study.

We would also like to recognize the contributions of various online academic communities and open-source platforms, whose resources, publications, and discussions greatly enriched our understanding of predictive modeling techniques and educational data mining. The availability of shared knowledge and collaborative tools played a crucial role in refining our methodology and ensuring the accuracy of our findings. Furthermore, we appreciate the students who willingly participated in surveys and interviews, providing valuable perspectives that added depth and context to our research. Their honest responses and shared experiences offered practical insights that enhanced the relevance and applicability of our study's outcomes

We also acknowledge the support of Ganpat university, which provided access to relevant datasets and computational resources essential for conducting our analysis. Special thanks are due to the faculty members and administrative staff who assisted with data collection and provided helpful insights into educational practices and student behaviour.

Finally, we are grateful to our peers, friends, and families for their unwavering encouragement and moral support during the research process. Their patience and motivation helped us persevere through challenges and maintain our focus on achieving meaningful results.

Sincerely,
Bhargav 22012531062
Prudhvi 22012531035
Murtuja 22012531062
Chandra 22012531056

ABSTRACT

In recent years, the rapid growth of digital payment systems such as online banking, credit/debit card transactions, and mobile wallets has significantly increased the convenience of financial transactions. However, this advancement has also led to a rise in fraudulent activities, posing serious threats to financial institutions and users. Detecting fraudulent transactions in real time has become a critical challenge due to the large volume of transactions and the evolving nature of fraud patterns.

This project, titled **"Online Payment Fraud Detection using Machine Learning,"** presents an intelligent and efficient solution to identify fraudulent transactions by leveraging machine learning techniques. The system is designed to analyze transaction data and detect suspicious activities based on patterns and anomalies in user behavior. The dataset used in this project contains various attributes such as transaction amount, payment method, merchant category, device usage, transaction time, and location-based features, which play a vital role in determining the authenticity of a transaction.

To ensure accurate predictions, the data undergoes several preprocessing steps, including data cleaning, handling missing values, encoding categorical variables using techniques like One-Hot Encoding, and feature scaling using StandardScaler. These steps help in improving the performance and reliability of the machine learning model. The processed data is then used to train a **Random Forest Classifier** model, which is well-suited for binary classification problems such as fraud detection. The model learns from historical transaction data and identifies patterns that distinguish fraudulent transactions from legitimate ones.

Once trained, the model is capable of predicting the probability of fraud for new incoming transactions. Based on a predefined threshold, the system classifies each transaction as either **fraudulent** or **legitimate**. If a transaction is identified as suspicious, the system generates an alert, enabling quick action to prevent potential financial loss. This automated approach significantly reduces the dependency on manual monitoring and increases the speed and accuracy of fraud detection.

The proposed system offers several advantages, including improved transaction security, reduced financial risk, scalability for handling large datasets, and the ability to adapt to new fraud patterns over time. Additionally, the system can be integrated with web-based applications using frameworks like **Fast API** to provide real-time fraud detection services.

In conclusion, this project demonstrates how machine learning can be effectively utilized to address the growing problem of online payment fraud. Future enhancements may include the use of advanced algorithms such as Gradient Boosting, or Deep Learning models, as well as real-time deployment in live financial systems to further improve accuracy and performance.

INDEX

Table of Contents

1. INTRODUCTION 1
   1.1 Solution Overview 1
   1.2 Key Features 1
   1.3 Benefits 2
2. PROJECT SCOPE 3
   2.1 Core Objective 3
   2.2 Project Scope Boundaries 3
   2.3 Benefits and Use Cases 4
   2.4 Challenges and Mitigation Strategies 5
3. LITERATURE SURVEY 6
4. SOFTWARE AND HARDWARE REQUIREMENTS 7
   4.1 Software Requirements 7
   4.2 Hardware Requirements 7
5. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS 8
   5.1 Functional Requirements 8
   5.2 Non-Functional Requirements 8
6. PROCESS MODEL 9
7. PROJECT PLANE 10
8. SYSTEM DESIGN 12
   8.1 Diagrams 12
   8.1.1 Use case Diagram 12
   8.1.2 Class Diagram 13
   8.1.3 Sequence diagram 14
   8.1.4 Activity diagram 15
   8.1.5 E-R Diagram 16
   8.2 List of tables, Table design, Data dictionary 17
   8.2.1 List of the tables 17
   8.2.2 Table Design 17
   8.2.3 Data Dictionary 18
   8.3 Design Strategy 18
   8.4 Data Flow Diagram: 19
   8.4.1 Level 0 DFD 19
   8.4.2 Level 1 DFD 19
   8.4.3 Level 2 DFD 20
   8.5 User Interface 21
9. IMPLEMENTATION DETAILS 24
10. TESTING 26
11. USER MANUAL 28
12. CONCLUSION AND FUTURE WORK 30
13. ANNEXURE 31

LIST OF FIGURES

Table of Contents

Use Case Diagram 12
Class Diagram 13
Sequence Diagram 14
Activity Diagram 15
E-R Diagram 5
DFD Level-0 19
DFD Level-1 20
DFD Level-2 20

List of Tables

Table of Contents

Users 17
Translation Logs 17
Chat History 17
Data Dictonary 18
Design Strategy 18
Sample test Causes and Results 26
Software Used 28
Glossary of terms and Abbreviations 31
About Tools and Technologies Used 32

INTRODUCTION

In today's digital era, online payment systems have become an integral part of daily life, enabling fast, secure, and convenient financial transactions. With the increasing use of credit cards, debit cards, mobile wallets, and online banking platforms, the volume of digital transactions has grown exponentially. However, this rapid growth has also led to a significant rise in fraudulent activities, making fraud detection a critical concern for financial institutions and users alike.

Traditional fraud detection methods, which rely on rule-based systems and manual monitoring, are no longer sufficient to handle the complexity and scale of modern fraud patterns. Fraudsters continuously evolve their techniques, making it difficult for static systems to detect new types of fraudulent behavior. As a result, there is a need for intelligent, adaptive, and automated systems that can analyze large volumes of transaction data and identify suspicious activities in real time.

This project, **"Online Payment Fraud Detection using Machine Learning,"** aims to address this challenge by developing a machine learning-based system capable of detecting fraudulent transactions efficiently. Machine learning algorithms can learn patterns from historical data and identify anomalies that may indicate fraudulent behavior. By leveraging these techniques, the system can improve detection accuracy and reduce false positives.

1.1 Solution Overview

The proposed system is designed to detect fraudulent transactions by analyzing transaction data using machine learning techniques. The workflow of the system includes data collection, preprocessing, model training, and prediction.

The system takes transaction data as input, processes it using data preprocessing techniques, and feeds it into a trained machine learning model. The model evaluates the transaction and predicts whether it is fraudulent or legitimate. Based on the prediction, appropriate actions such as alert generation can be taken.

1.2 Key Features

- **Data Preprocessing Pipeline:**
  - Handling missing and inconsistent transaction data.
  - Encoding categorical features using One-Hot Encoding.
  - Scaling numerical data such as transaction amount using StandardScaler.
  - Removing noise and preparing clean data for model training.

- **Machine Learning Model:**
  - Implementation of Logistic Regression for fraud classification.
  - Learns patterns from historical transaction data.
  - Provides probability-based fraud detection results.
  - Efficient and interpretable model for binary classification.

- **Fraud Prediction System:**
  - Real-time analysis of incoming transaction data.
  - Classification of transactions as fraudulent or legitimate.
  - Uses threshold-based decision making for fraud detection.
  - Quick identification of suspicious activities.

- **Scalability:**
  - Handles large volumes of transaction data efficiently.
  - Can be extended with advanced machine learning models.
  - Supports retraining with updated datasets.
  - Suitable for growing digital payment systems.

- **Automation:**
  - Fully automated fraud detection process.
  - Reduces manual monitoring and human errors.
  - Continuous transaction analysis without interruption.
  - Fast and reliable prediction system.

1.3 Benefits

- **Improved Transaction Security:**
  Detects fraudulent transactions accurately and enhances the overall safety of online payment systems.
- **Reduction in Financial Losses:**
  Identifies fraudulent activities early and helps prevent financial losses for users and institutions.
- **Fast and Efficient Processing:**
  Provides real-time fraud detection with quick response to suspicious transactions.
- **Automation of Fraud Detection:**
  Eliminates manual monitoring by automatically analyzing and detecting fraudulent transactions.
- **Scalability and Flexibility:**
  Efficiently handles large volumes of data and adapts to evolving fraud patterns.
- **Better Decision Making:**
  Provides probability-based insights that help in evaluating transaction risk and taking timely actions.
- **High Accuracy in Detection:**
- Uses machine learning algorithms to achieve accurate classification of fraudulent and legitimate transactions.
- **Time-Saving System:**
  Reduces the time required for analyzing transactions by providing instant fraud detection results.
- **User Trust and Confidence:**
  Builds trust among users by ensuring secure and reliable online payment experiences.
- **Adaptability to New Fraud Patterns:**
  Continuously improves by learning from new data and adapting to evolving fraud techniques.
- **Cost Efficiency:**
  Reduces operational costs by minimizing manual fraud investigation efforts.
- **Easy Integration:**
  Can be integrated with existing payment systems and applications with minimal modifications.
- **Continuous Monitoring:**
  Enables 24/7 monitoring of transactions to detect suspicious activities without interruption.

2. PROJECT SCOPE

The scope of the Online Payment Fraud Detection project defines the boundaries, goals, functionalities, and technologies involved in its development and deployment.

2.1 Core Objective:

The core objective of Real time speech translation is to bridge language barriers by providing a real-time, AI-powered translation and immersive language learning platform. It enables users to communicate across languages instantly through advanced translation tools, automatic language detection, and text-to-speech features. Real time speech translation also offers interactive language practice with an AI chatbot that simulates real-world conversations, providing feedback and corrections to help users improve their fluency. With personalized user dashboards, secure authentication, and a modern, responsive interface, Real time speech translation empowers users to learn and communicate confidently while ensuring a seamless, engaging, and private learning experience.

2.2 Project Scope Boundaries:

- **In-Scope:**
- **Dataset Processing:**
- Loading, cleaning and preprocessing the Pay Sim synthetic financial dataset for model training and evaluation.
- **Feature Engineering:**
- One-Hot encoding for the transaction type column, StandardScaler for numerical features, and class balancing using class weighting.
- **Model Training:**
- Training a logistic regression classifier for binary fraud classification (fraud vs legitimate transactions).
- **Model Evaluation:**
- Evaluating the model using accuracy, precision, recall, F1-score and confusion matrix on the test spilt.
- **Web Application:**
- A Flask-based web interface for users to enter transaction details and receive real-time fraud predictions.
- **Deployment:**
- Local deployment of the web application for demonstration and testing purposes.

- **Out-of-Scope:**
- **Real-Time API Integration:**
- No live connection to actual banking transaction streams is implemented in the current scope.
- **User Authentication:**
- The web application does not include a user login or registration system at this stage.
- **Cloud Deployment:**
- The project is locally deployed; cloud hosting on AWS, GCP, or Heroku is a future enhancement.
- **Advanced Model Ensembles:**
- Deep learning models or complex ensemble methods are reserved for future phases.

➢ **Mobile Application:**

- No native iOS or Android app is developed; the system is entirely web-based.

2.3 Benefits and Use Cases:

▪ **Benefits:**

- Reduces financial losses by accurately flagging fraudulent transactions before they are processed by payment systems.
- Automates the fraud detection process, minimizing the need for manual review of large transaction volumes.
- Provides a scalable solution that can handle growing datasets with minimal architectural changes.
- Serves as a solid foundation for advanced fraud detection using ensemble methods or deep learning in the future.
- Demonstrates practical application of machine learning in a real-world fintech problem domain relevant to financial institutions.

▪ **Use Cases:**

- **Financial Institutions:**

  Banks and payment processors can use the model to screen transactions before approving them.

- **E-Commerce Platforms:**

  Online retail platforms can integrate fraud detection to protect buyers and sellers from unauthorized transactions.

- **Fintech Startups:**

  Companies building payment infrastructure can embed this system for baseline fraud screening.

- **Internal Audits:**

  Finance teams can batch-process transaction logs to identify suspicious historical activity.

2.4 Challenges and Mitigation Strategies:

1. Class Imbalance:

- **Challenge:** Fraudulent transactions represent less than 0.2% of total transactions in the Pay Sim dataset.
- **Mitigation:** Applied class_weight="balanced" in the Logistic Regression model to handle the heavily imbalanced dataset and ensure adequate weight is given to the fraud class during training.

2. Feature Selection:

- **Challenge:** Not all features in the dataset contribute equally to fraud detection performance.
- **Mitigation:** Performed correlation analysis and used domain knowledge to select the most relevant features (transaction type, amount, and balance values for origin and destination accounts).

3. Model Generalization:

- **Challenge:** Overfitting on training data can reduce performance on new, unseen transactions.
- **Mitigation:** Applied an 80:20 train-test split and evaluated the model on the held-out test set to ensure consistent generalization.

4. Data Preprocessing Complexity:

- **Challenge:** Raw transaction data contains categorical variables and features with widely varying numerical scales.
- **Mitigation:** Implemented a structured preprocessing pipeline using pandas One-Hot Encoding and scikit-learn's StandardScaler via Column Transformer.

5. Web App Input Validation:

- **Challenge:** Incorrect or malicious user inputs can destabilize the prediction endpoint.
- **Mitigation:** Added server-side input validation and error handling in the Flask backend to ensure stable and reliable prediction responses.

3. LITERATURE SURVEY

The literature survey provides an overview of existing research, tools, and techniques related to online payment fraud detection using machine learning. It highlights the strengths and limitations of prior approaches and establishes the motivation for the methodology adopted in this project.

1. Rule-Based Fraud Detection Systems:

- Traditional fraud detection relied on static rule sets defined by domain experts for example, flagging any transaction exceeding a fixed monetary threshold or originating from an unusual geography.
- While interpretable and auditable, these systems are inflexible, generate high false-positive rates, and cannot adapt to evolving fraud tactics.

Key Insight: Static rule engines cannot keep pace with sophisticated and dynamic fraud patterns, making intelligent adaptive systems essential.

2. Logistic Regression for Fraud Detection:

- Logistic Regression is one of the most widely studied algorithms for binary classification in fraud detection.
- Studies using the UCI credit card fraud dataset demonstrate that Logistic Regression achieves competitive baseline performance when features are properly preprocessed and class imbalance is addressed using techniques like class weighting or SMOTE.

Key Insight: Logistic Regression provides a transparent, interpretable, and computationally efficient baseline that is well-suited for fraud classification as a first-step model.

3. Decision Trees and Ensemble Methods:

- Ensemble methods such as Random Forest and Gradient Boosting have been shown to significantly outperform single classifiers for fraud detection by combining multiple weak learners.
- Sahin et al. (2013) demonstrated that Random Forest achieves higher recall for fraud cases on imbalanced datasets. XGBoost and LightGBM are now widely used in industry-grade fraud systems.

Key Insight: Ensemble models provide improved accuracy at higher computational cost and reduced interpretability compared to Logistic Regression.

4. Deep Learning Approaches:

- Neural networks and LSTM models have been applied to sequential transaction data to capture temporal behavioral patterns.
- While powerful, these approaches require significantly larger datasets, more computational resources, and are harder to interpret and deploy.

Key Insight: Deep learning is promising for large-scale real-time fraud detection but introduces complexity in implementation, training, and regulatory compliance (explainability requirements).

5. PaySim Synthetic Dataset:

- The PaySim dataset, synthesized by Lopez-Rojas et al. (2016), simulates mobile money transactions based on real transaction logs from an African mobile money service.
- It provides labeled fraud/legitimate records and is widely used as a benchmark for supervised fraud detection research

Key Insight: PaySim provides a practical, reproducible, and publicly available dataset for evaluating fraud detection models without exposing sensitive real-world financial data.

6. Handling Class Imbalance in Fraud Datasets:

- Fraud datasets are inherently highly imbalanced — legitimate transactions outnumber fraudulent ones by ratios of 500:1
- or more. Chawla et al. (2002) introduced SMOTE (Synthetic Minority Oversampling Technique) as an effective
- approach for improving minority class recall. Cost-sensitive learning (class weighting) is a simpler and equally effective
- alternative.

Key Insight: Addressing class imbalance is critical to achieving high recall for the fraud class; without it, models predict all transactions as legitimate and miss actual fraud.

4. SOFTWARE AND HARDWARE REQUIREMENTS

4.1 Software Requirements:

- **Programming Language and Runtime**
  - Python 3.11. +
  - HTML, CSS, JS.

- **Machine Learning and Data Science Libraries**
  - Pandas.
  - Numpy.
  - Scikit-learn.
  - Matplotlib / Seaborn.
  - Joblib.
  - Imbalanced-learn. (Optional)

- **Web Framework and Deployment**
  - Flask.
  - Jinja2.
  - Werkzeug
  - Gunicorn. (Optional)

- **Development Tools**
  - Jupyter Notebook / VS Code.
  - Git / GitHub.
  - Postman.
  - Browser DevTool.

4.2 Hardware Requirements:

For End Users (Web Access)

- **Device**: Desktop, laptop, or tablet with a modern web browser
- **Browser:** Google Chrome, Mozilla Firefox, Microsoft Edge or Safari etc.,
- **RAM**: Minimum 4 GB
- **Internet**: Local network or internet access to reach the Flask server.

For Developers

- **Processor**: Quad-core (Intel i5 / AMD Ryzen 5 or better)
- **RAM**: Minimum 8 GB (16 GB recommended)
- **Storage**: 2 GB+ free space for local development
- **Operating System**: Windows 10/11, macOS, or Linux
- **Internet**: Required for installing libraries and GitHub repository access.

5. FUNCTIONAL AND NON-FUNCTIONAL REQUIREMENTS

5.1 Functional Requirements:

These are the core features and behaviors the system must perform:

1. Data Loading and Preprocessing:
   - The system must load the PaySim CSV dataset, perform data cleaning, drop irrelevant columns (nameOrig, nameDest), encode the "type" column using One-Hot Encoding, and scale numerical features using StandardScaler.

2. Model Training:
   - The system must train a Logistic Regression classifier on the preprocessed fraud transaction dataset and save the trained model and preprocessing pipeline using joblib for reuse in prediction.

3. Fraud Prediction:
   - Given input transaction features (type, amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest), the system must predict whether a transaction is Fraudulent or Legitimate and display the result.

4. Web Interface-Input Form:
   - The Flask web application must provide an HTML form where users can enter transaction details manually, including transaction type (dropdown) and numerical balance fields.
5. Web Interface-Result Display:
   - The system must clearly display the fraud prediction result on the result page, indicating Fraudulent or Legitimate with an appropriate visual indicator.

6. Model Evaluation:
   - The system must evaluate model performance using accuracy, precision, recall, F1-score, and confusion matrix, displayed during the training and analysis phase in the Jupyter Notebook.
7. Error Handling:
   - The web application must validate user inputs server-side and handle invalid, missing, or out-of-range values gracefully by displaying clear error messages to the user.

5.2 Non-Functional Requirements:

These define how the system performs rather than what it does:

1. **Performance**
   - The model prediction must return a result within 1–2 seconds of form submission under normal local operating conditions. Training on the full PaySim dataset must complete within a reasonable time on standard developer hardware.

2. **Scalability**
   - The preprocessing pipeline and model must support retraining with larger or updated datasets without requiring architectural changes to the system.

3. **Usability**
   - The web interface must be clean, intuitive, and operable by non-technical users without requiring programming knowledge or installation of any software on the user side.

4. **Security**
   - API keys, sensitive configuration files, and serialized model files must not be committed to version control. A .gitignore file must exclude sensitive and environment-specific files from the GitHub repository.

5. **Accuracy:**
   - The trained Logistic Regression model must achieve at least 85% overall accuracy, with particular emphasis on recall for the fraud class (minority class) to minimize missed fraudulent transactions.

6. **Maintainability:**
   - The codebase must be modular and well-documented, with separate files for data preprocessing, model training, and the web application, supporting easy updates and debugging.

7. **Portability:**
   The application must run on Windows, macOS, and Linux with standard Python package installations. The web interface must function correctly on all modern web browsers.

8. **Compatibility:**
   - Compatible with multiple screen sizes (desktop, tablet); mobile responsiveness without being a native mobile app.

6. PROCESS MODEL

The development of the Online Payment Fraud Detection system follows an Agile Incremental Process Model, which is best suited for data science and machine learning projects where requirements evolve progressively as dataset analysis deepens and model performance insights are gained across development cycles.

Increment 1 — Data Acquisition and Exploration

In this phase, the PaySim synthetic financial dataset is acquired and loaded into a Jupyter Notebook development environment. Exploratory Data Analysis (EDA) is performed to understand the distribution of transaction types (CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER), transaction amount ranges, fraud ratios, and feature correlations. Visualizations such as histograms, box plots, count plots, and correlation heatmaps are generated to identify patterns. The primary objective is to develop a deep understanding of the data structure, class imbalance ratio, and feature relevance before any modeling begins.

Increment 2 — Data Preprocessing and Feature Engineering

The raw dataset is preprocessed through a structured pipeline. Irrelevant identifier columns (nameOrig, nameDest) and the isFlaggedFraud column are dropped. The categorical "type" feature is encoded using pandas get_dummies() or scikit-learn's OneHotEncoder. Numerical features (amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest) are scaled using StandardScaler. Class imbalance is addressed by setting class_weight="balanced" in the Logistic Regression model. The cleaned dataset is split into 80% training and 20% testing sets using scikit-learn's train_test_split with a fixed random state for reproducibility.

Increment 3 — Model Training and Evaluation

A Logistic Regression classifier is trained on the preprocessed training data with class_weight="balanced" to handle the severe fraud class imbalance. The model is evaluated on the held-out test set using multiple metrics: accuracy, precision, recall, F1-score, and the confusion matrix. Classification reports are analyzed to identify performance on the minority fraud class specifically. The trained model and the fitted preprocessing pipeline are serialized using joblib.dump() and saved as .pkl files for later use in the Flask web application without retraining.

Increment 4 — Web Application Development and Testing

A Flask web application is built that loads the serialized model and preprocessing pipeline at startup using joblib.load(). An HTML/CSS front-end form allows users to enter transaction type and numerical balance values. The Flask backend receives the POST request, applies the same preprocessing transformations as during training, and returns a Fraudulent or Legitimate prediction rendered on the result page. Black box testing is performed across various valid and invalid transaction inputs to validate correct system behavior and error handling.

7. PROJECT PLANE

Week	Timeline	Goals / Activities	Deliverables
Week 1	Days 1–7	Finalize requirements; select dataset (PaySim); plan system architecture; review related literature on ML fraud detection	SRS Document; Dataset selected
Week 2	Days 8–14	Load and explore PaySim dataset; perform EDA; analyze fraud distribution, transaction types, and feature correlations; generate visualizations	EDA Report; Data understanding complete
Week 3	Days 15–21	Data cleaning; One-Hot Encoding for type column; StandardScaler for numerical features; train-test split; handle class imbalance with class weighting	Preprocessing pipeline (scikit-learn)
Week 4	Days 22–28	Train Logistic Regression model; evaluate with accuracy, precision, recall, F1-score, and confusion matrix on the test set	Trained model saved as .pkl via joblib
Week 5	Days 29–35	Analyze model results; tune hyperparameters (C value, solver, max_iter); document findings; prepare model evaluation report	Model evaluation and tuning report
Week 6	Days 36–42	Build Flask web application; design HTML/CSS input form for transaction details; integrate trained model for real-time predictions	Working Flask web app (local deployment)
Week 7	Days 43–49		Fully tested and debugged system
Week 8	Days 50–56	Prepare project documentation; write user manual and installation guide; finalize GitHub repository with README and requirements.txt	Complete documentation + GitHub repo
Week 9	Days 57–60	Final demo preparation; create project presentation; buffer time for final fixes, improvements, and submission readiness	Final working product ready for demo

Summary

- ML Pipeline: Weeks 2–5
- Web Application: Weeks 6–7
- Testing and Documentation: Weeks 7–8
- Final Demo: Week 9

8. SYSTEM DESIGN

8.1 Diagrams

8.1.1 Use case Diagram:

The Use Case Diagram for this project represents the interaction between the User (external actor) and the core functionalities of the Online Payment Fraud Detection system. The primary use cases include: entering transaction details via the web form, receiving fraud/legitimate prediction, and the system internally performing preprocessing, model inference, and result generation. The system actor (Fraud Detection System) encapsulates all internal processes.

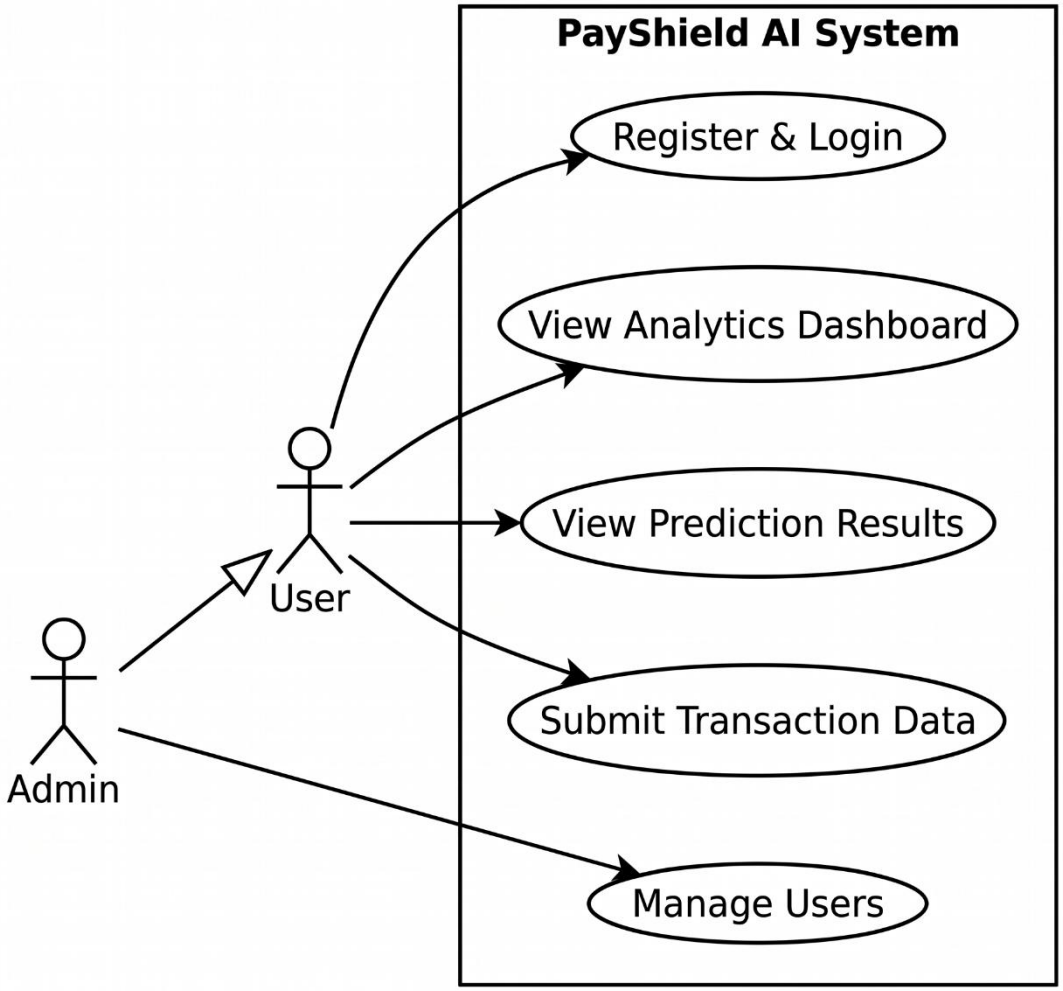


Figure 1 Use Case Diagram

8.1.2 Class Diagram:

The Class Diagram defines the structural blueprint of the fraud detection system. Key classes include: TransactionData (attributes: type, amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest), DataPreprocessor (methods: encode_categorical(), scale_features(), clean_data(), split_data()), FraudDetectionModel (methods: train(), predict(), evaluate(), save(), load()), and FlaskApp (routes: index(), predict_fraud(), display_result()).

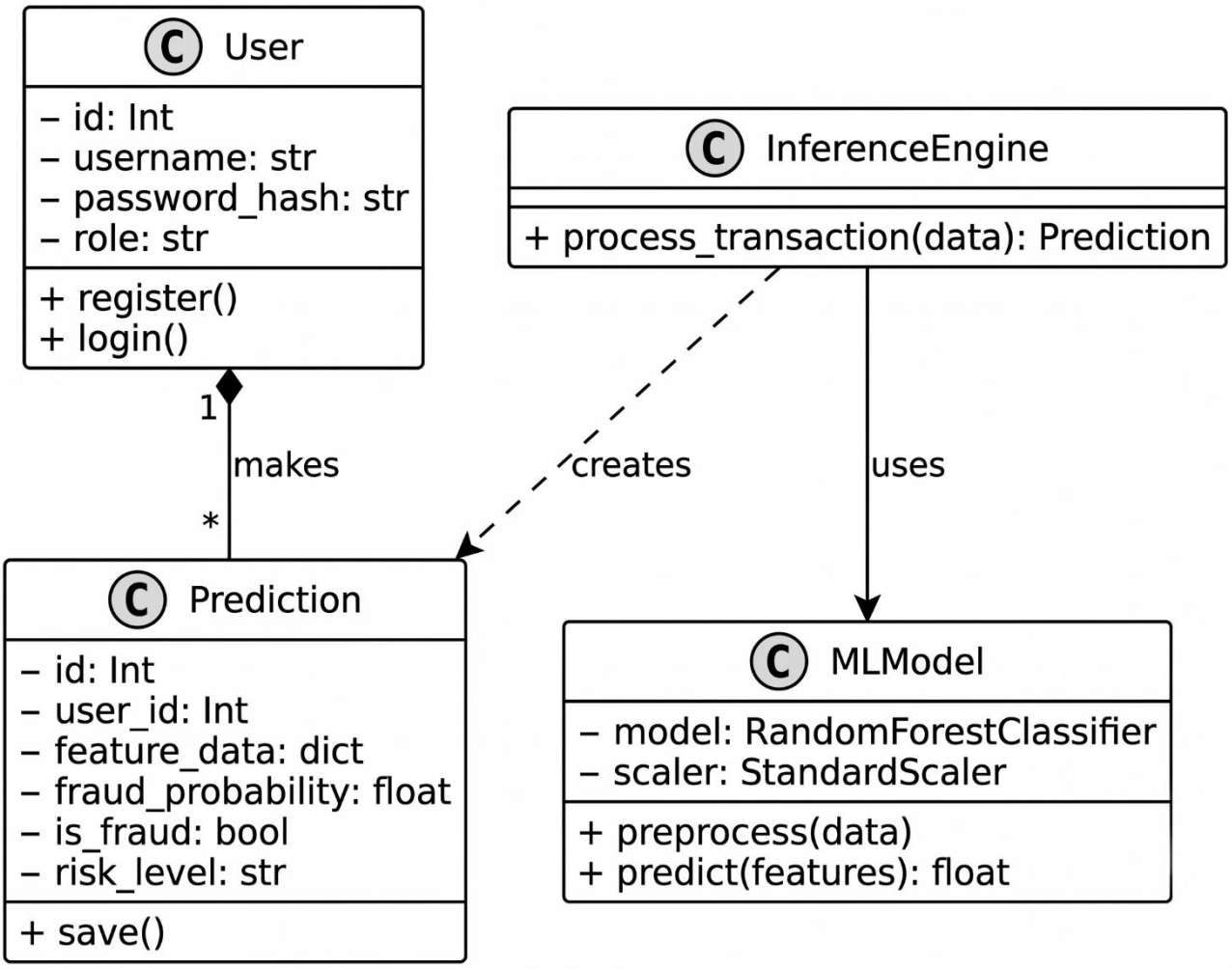


Figure 2 Class Diagram

8.1.3 Sequence diagram:

The Sequence Diagram illustrates the interaction flow when a user submits a transaction for fraud prediction. Sequence: User fills and submits the HTML form → Flask receives the POST request → Preprocessing pipeline (loaded from joblib) transforms the input → Logistic Regression model performs inference → Prediction result ("Fraud" or "Legitimate") is returned to Flask → Flask renders the result template → User sees the prediction on the browser.

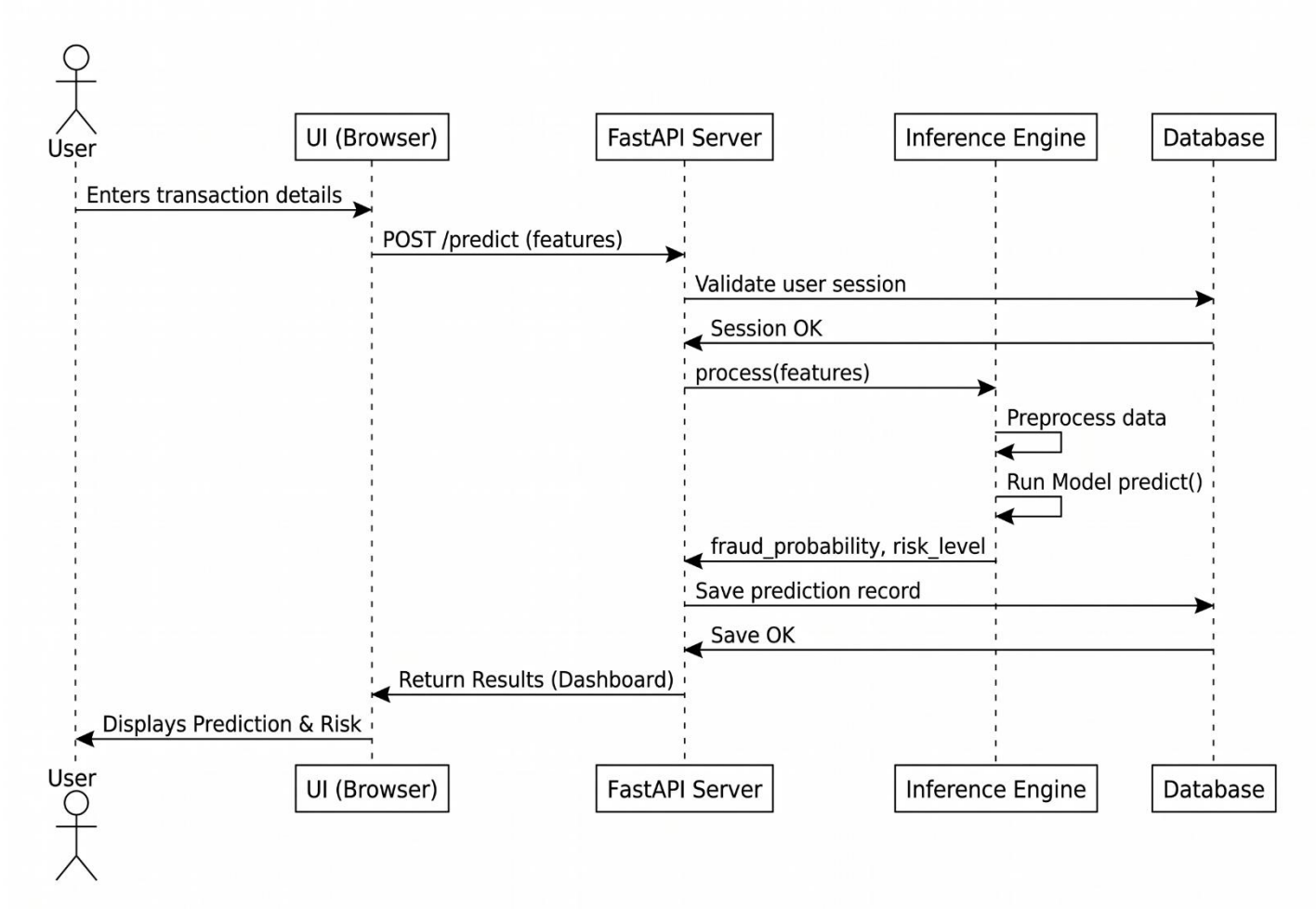


Figure 3 Sequence Diagram

8.1.4 Activity diagram:

The Activity Diagram illustrates the complete workflow from user input to fraud prediction output. The flow starts with the user opening the web application and entering transaction details. The system validates the input — if invalid, an error message is shown; if valid, the preprocessing pipeline is applied, the model performs inference, and the prediction (Fraudulent or Legitimate) is displayed on the result page.

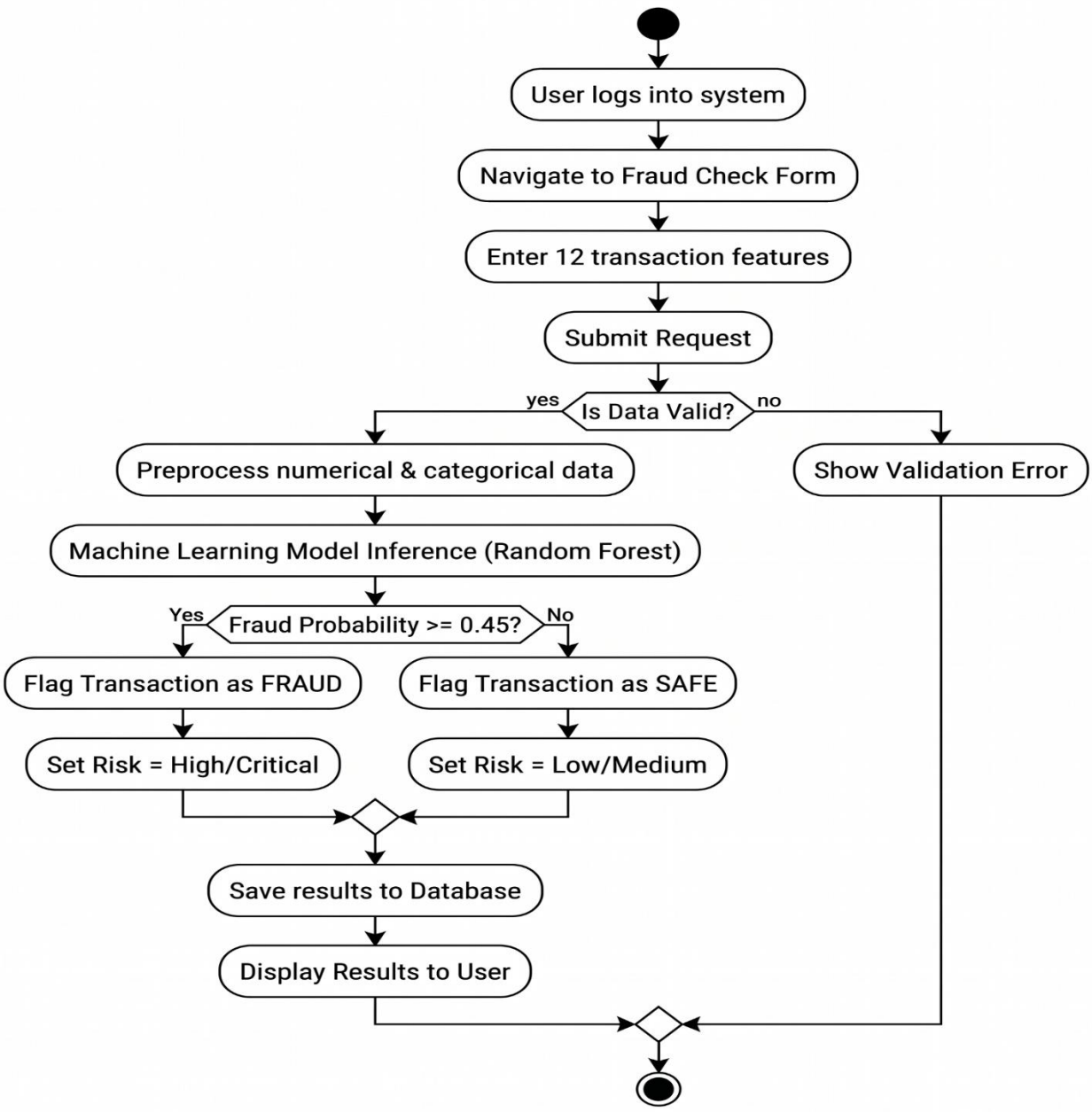


Figure 4 Activity Diagram

8.1.5 E-R Diagram:

The Entity-Relationship Diagram illustrates the key data entities: Transaction (attributes: transaction_id, type, amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest, isFraud) and PredictionLog (attributes: log_id, transaction_id, predicted_label, confidence_score, timestamp). A Transaction entity has a one-to-one relationship with a PredictionLog entry recording the model's output.

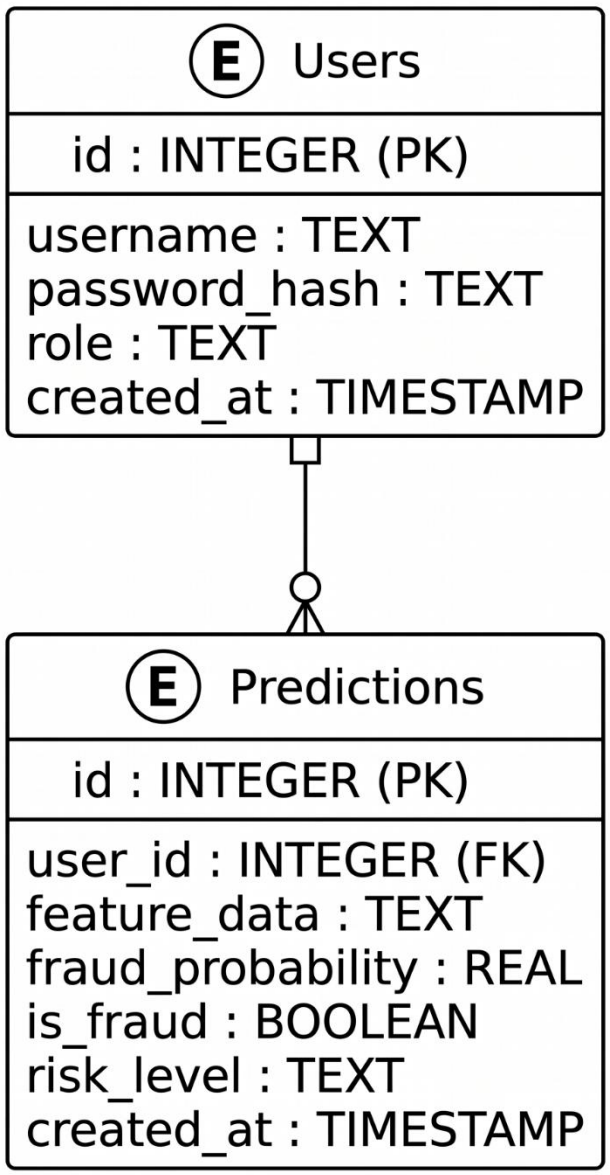


Figure 5 E-R Diagram

8.2 List of tables, Table design, Data dictionary:

8.2.1 List of the tables:

Table Name	Description
users	Stores user profile data (synced via Clerk)
translations	Stores raw transaction records loaded from the PaySim dataset for preprocessing and training
prediction_logs	Stores prediction results: input features, predicted label, confidence score, and timestamp

8.2.2 Table Design:

Table Design Users:

Column Name	Data Type	Description	Constraints
user_id	VARCHAR	Unique ID from Clerk	Primary Key
name	VARCHAR	Full name of the user	Not Null
email	VARCHAR	User's email	Unique, Not Null
created_at	DATETIME	Account creation timestamp	Default: now()

Table Design: transactions:

Column Name	Data Type	Description	Constraints
transaction_id	INT	Auto-incremented unique identifier for each transaction record	Primary Key
type	VARCHAR(20)	Type of transaction: CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER	Not Null
amount	FLOAT	Transaction amount in local currency units	Not Null
oldbalanceOrg	FLOAT	Balance of the origin account before the transaction was initiated	Not Null
newbalanceOrig	FLOAT	Balance of the origin account after the transaction was completed	Not Null
oldbalanceDest	FLOAT	Balance of the destination account before the transaction	Not Null
newbalanceDest	FLOAT	Balance of the destination account after the transaction	Not Null
isFraud	INT (0/1)	Target label: 1 if transaction is fraudulent, 0 if legitimate	Not Null

Table Design: prediction_logs

Column Name	Data Type	Description	Constraints
log_id	INT	Unique log entry identifier	Primary Key
type	VARCHAR(20)	Transaction type submitted via the web form	Not Null
amount	FLOAT	Transaction amount submitted for prediction	Not Null
predicted_label	VARCHAR(15)	Model prediction output: "Fraudulent" or "Legitimate"	Not Null
confidence_score	FLOAT	Predicted probability score from the Logistic Regression model	Not Null
timestamp	DATETIME	Date and time when the prediction was generated	Default: now()

8.2.3 Data Dictionary:

Field Name	Meaning / Purpose
type	Transaction category: CASH-IN, CASH-OUT, DEBIT, PAYMENT, or TRANSFER — encoded as One-Hot features
amount	Monetary value of the transaction in local currency; key fraud indicator
oldbalanceOrg	Origin account balance prior to the transaction; used to detect balance inconsistencies
newbalanceOrig	Origin account balance after the transaction; compared with oldbalanceOrg to verify consistency
oldbalanceDest	Destination account balance before the transaction; TRANSFER/CASH-OUT fraud often leaves this unchanged
newbalanceDest	Destination account balance after the transaction; zero values are a strong fraud indicator
isFraud	Binary target label: 1 = Fraudulent transaction, 0 = Legitimate transaction
predicted_label	Output of the Logistic Regression model for a given transaction input via the web app
confidence_score	Probability score (0.0 to 1.0) output by model.predict_proba() for the fraud class
timestamp	Server-side timestamp of when the fraud prediction was generated and logged

8.3 Design Strategy:

Architecture Type: 2-Tier Architecture
Architecture Type: 2-Tier Architecture (ML Pipeline + Web Application Layer)

Tiers Overview:

Tier	Components
ML Tier	Python scripts for data loading, EDA, preprocessing (pandas, scikit-learn), Logistic Regression model training, evaluation, and joblib serialization of model + pipeline
Application Tier	Flask web application: loads serialized model + preprocessor; accepts user input via HTML form; applies preprocessing; calls model.predict(); renders result to user

Data Flow:
1. User enters transaction details in the Flask HTML form.
2. Flask validates and collects form input.
3. joblib-loaded preprocessing pipeline applies One-Hot Encoding and StandardScaler.
4. Logistic Regression model performs inference.
5. Prediction result ("Fraudulent" or "Legitimate") is rendered on the result page for the user to see.

Benefits of 3-Tier Architecture:
- **Modularity**: Separates the machine learning logic and user interface, making the system easier to develop, manage, and update.
- **Scalability**: Allows the system to handle increasing data and users by independently scaling the application and model components.
- **Security**: Ensures sensitive data and model logic are handled in the backend, reducing exposure to unauthorized access.

8.4 Data Flow Diagram:

8.4.1 Level 0 DFD:

Represents the entire Online Payment Fraud Detection system as a single process. External entity: User. Input data flow: Transaction details (type, amount, balance values). Output data flow: Fraud / Legitimate prediction result. The system internally processes all transaction data and returns a binary classification output.

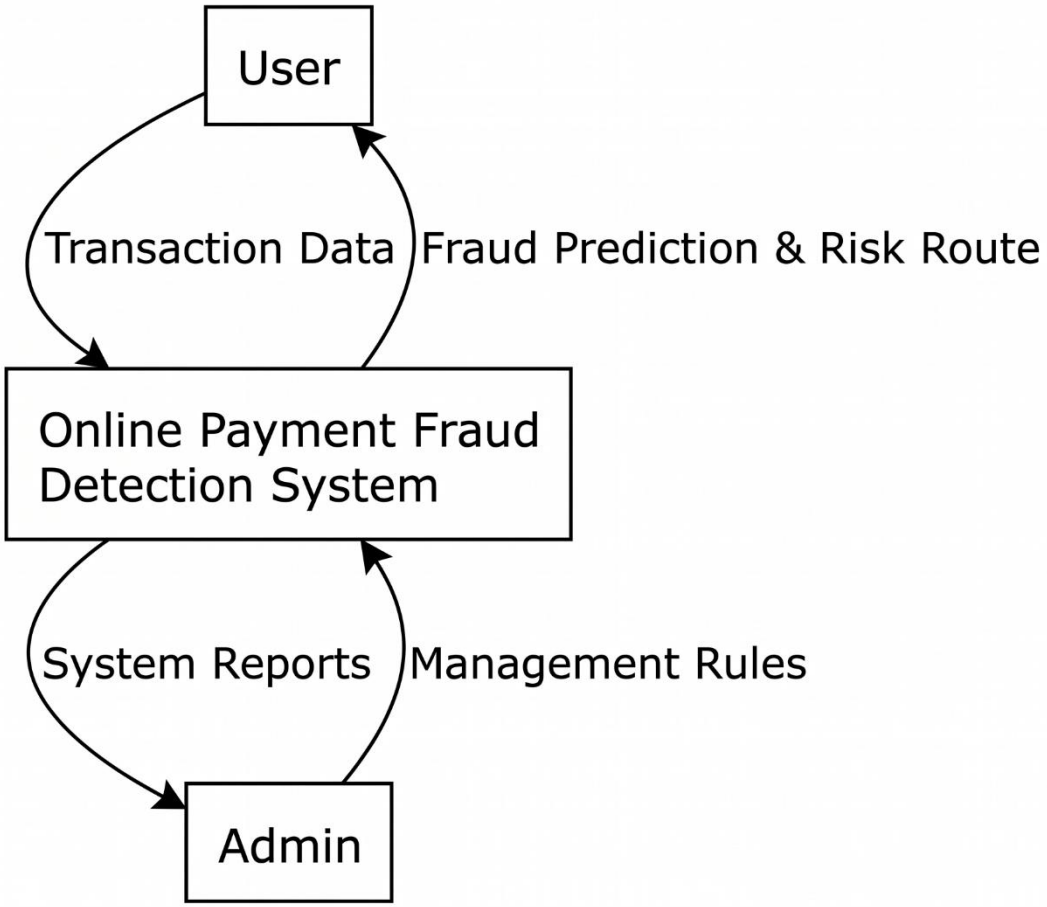


Figure 6 DFD Level 0

8.4.2 Level 1 DFD:

Breaks down the system into four major sub-processes: (1) Input Validation — validates user-submitted transaction details for completeness and valid ranges; (2) Data Preprocessing — applies One-Hot Encoding and StandardScaler transformations to the input; (3) Fraud Prediction — runs the trained Logistic Regression model on the preprocessed feature vector; (4) Result Display — returns and renders the prediction result (Fraudulent/Legitimate) to the user.

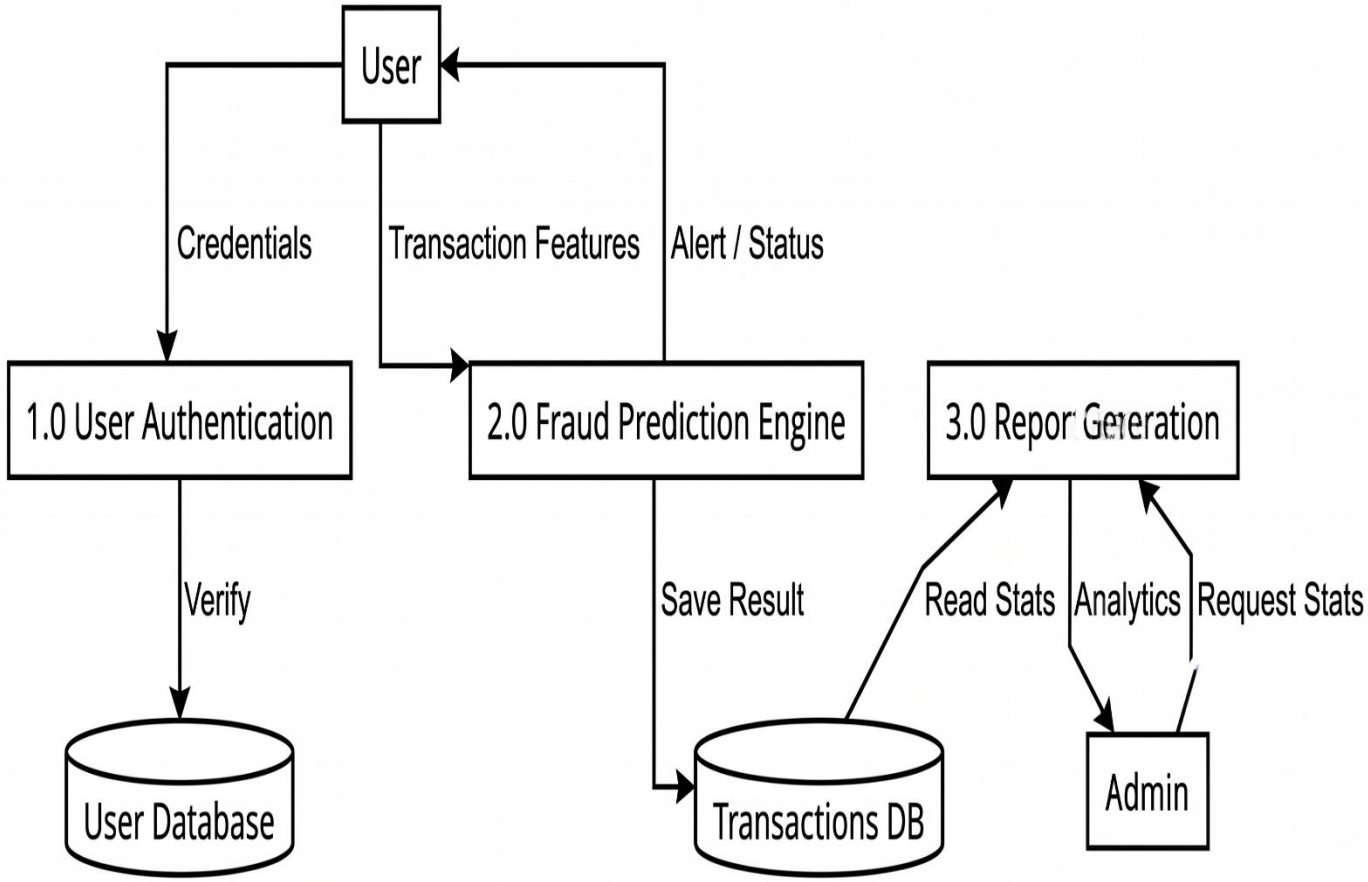


Figure 7 DFD Level 1

8.4.3 Level 2 DFD:

Further decomposes the Data Preprocessing sub-process into: categorical encoding (One-Hot Encoding for the transaction type field), numerical scaling (StandardScaler for amount and balance features), and feature vector assembly (combining encoded and scaled features into the model input). Also decomposes Fraud Prediction into: model loading from the joblib .pkl file, logistic regression probability computation (model.predict_proba()), and threshold-based binary classification (fraud if probability >= 0.5).

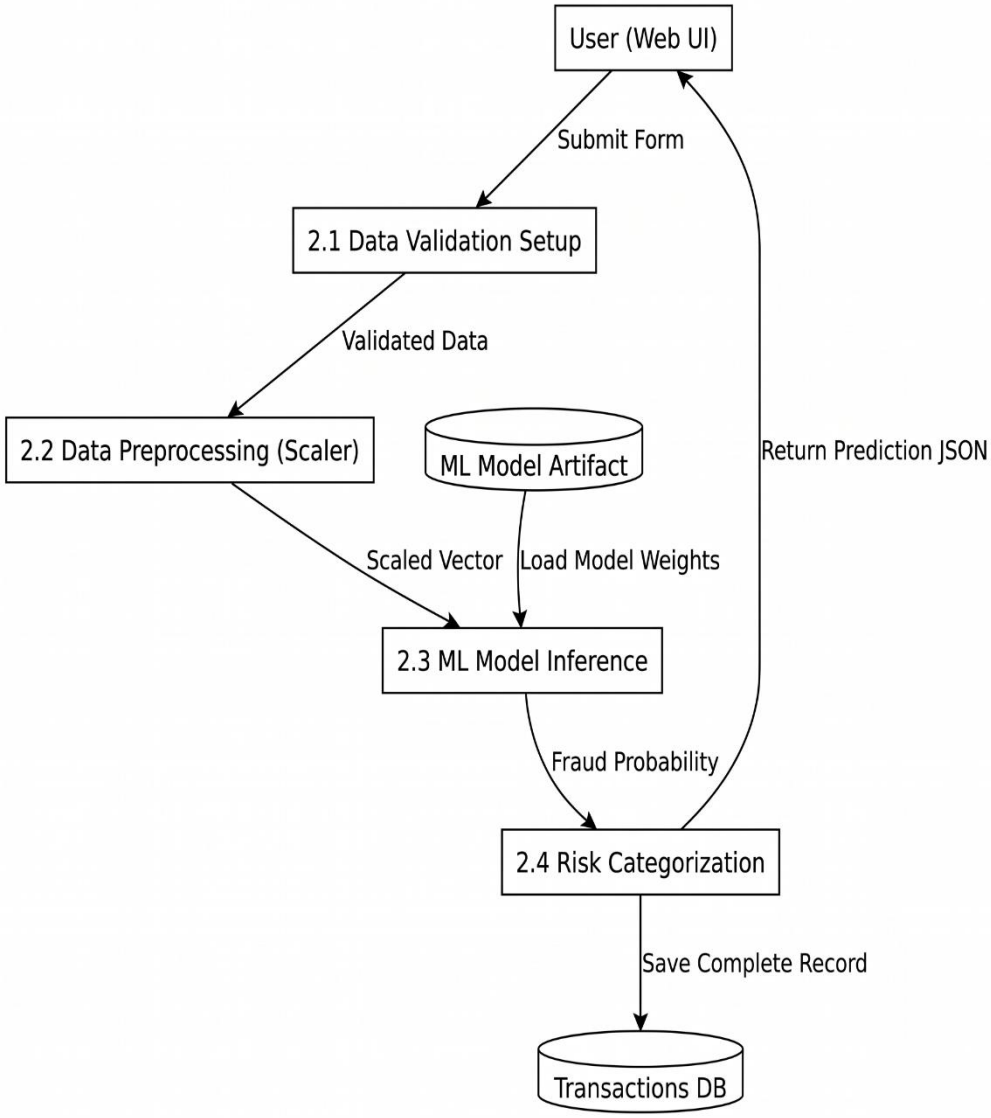
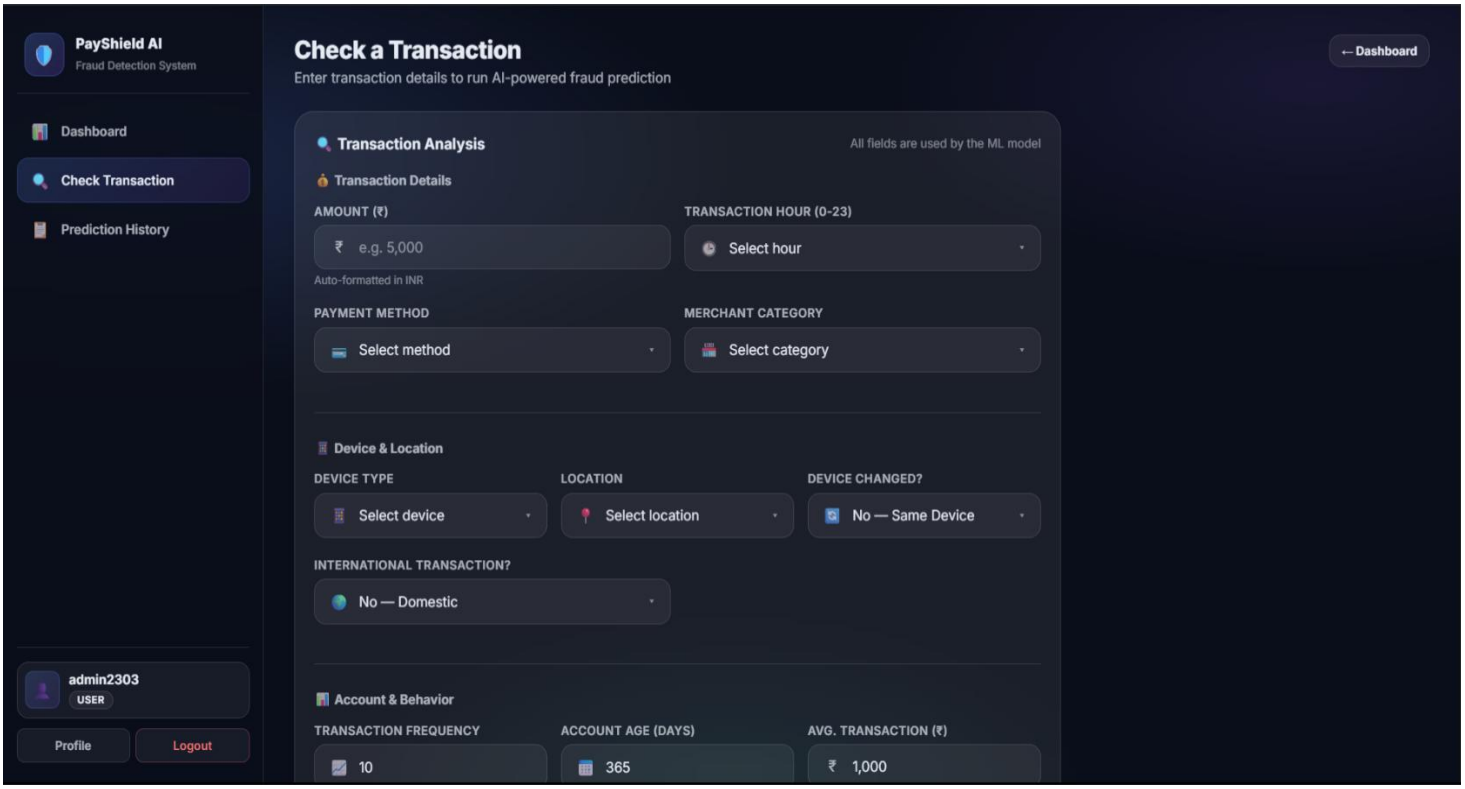


Figure 8 DFD Level 2

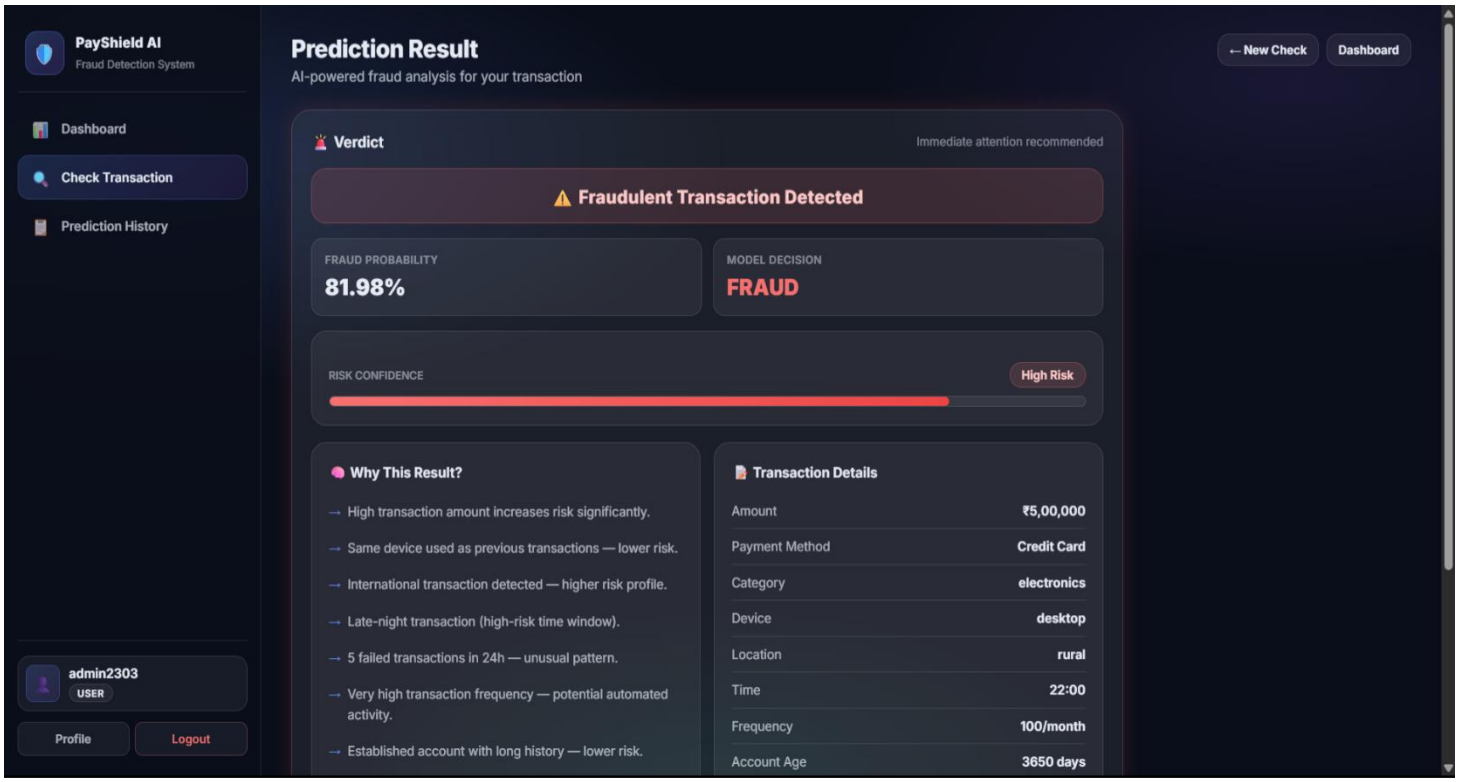
8.5 User Interface:

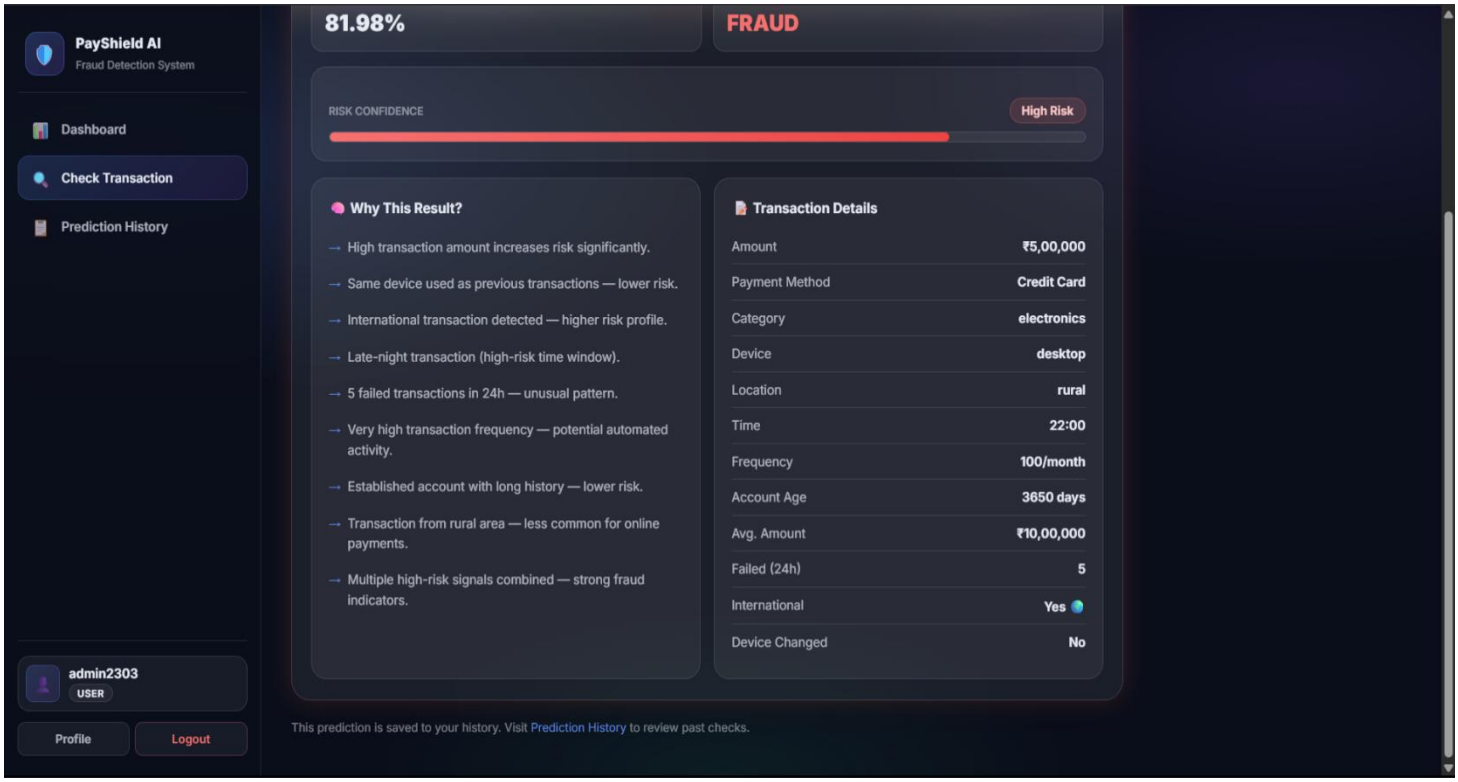
The web application interface consists of two pages: the Input Form Page and the Result Page.

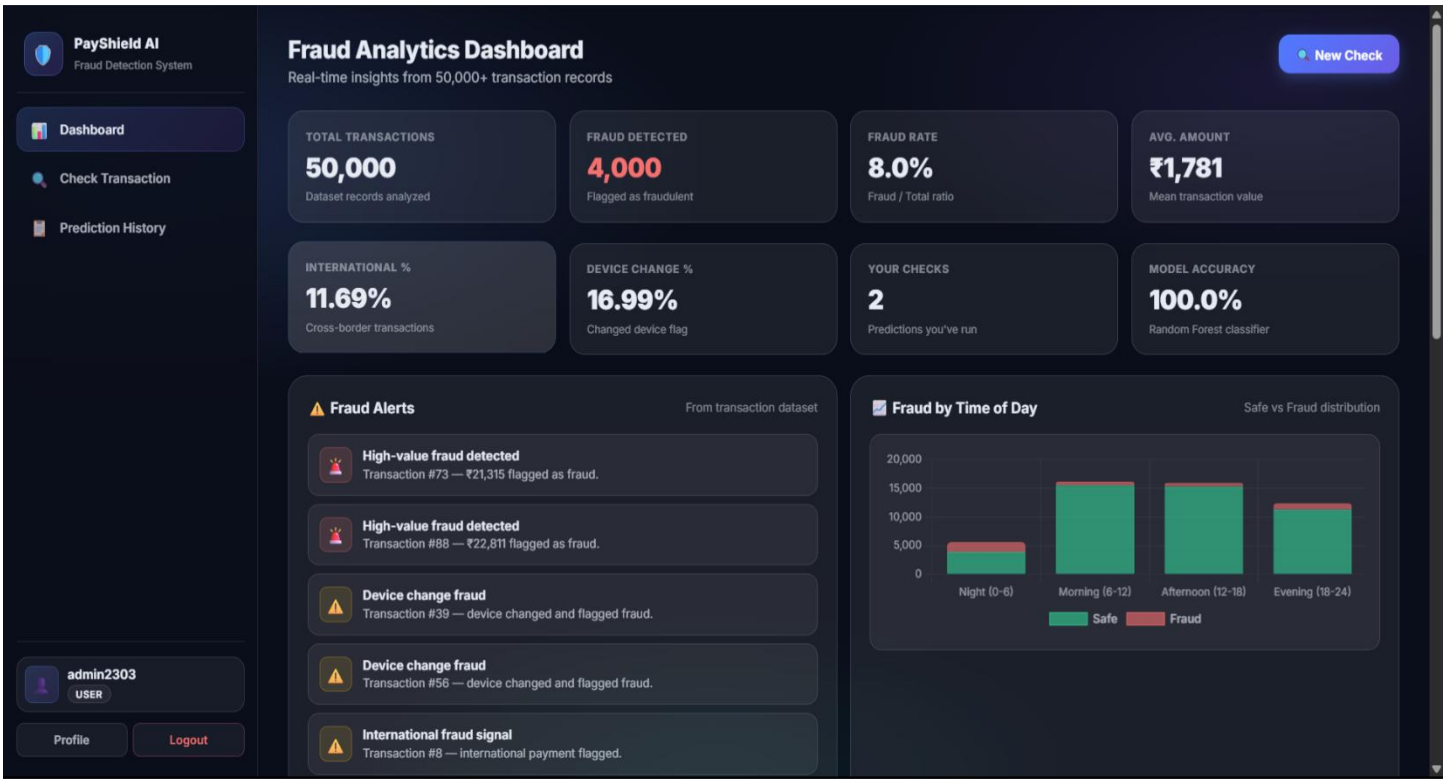


The Input Form Page presents a clean, structured HTML form where the user can select the transaction type from a dropdown menu (CASH-IN, CASH-OUT, DEBIT, PAYMENT, TRANSFER) and enter numerical values for transaction amount, origin account old balance, origin account new balance, destination account old balance, and destination account new balance. A "Predict" button submits the form data to the Flask /predict route.

The Result Page displays the fraud prediction outcome returned by the model clearly and prominently. A "Fraudulent" prediction is displayed with a red alert, while a "Legitimate" prediction is shown with a green confirmation. The confidence score (probability) is also displayed to indicate the model's certainty. A "Check Another Transaction" button returns the user to the input form


PayShield AI
Fraud Detection System
Dashboard
Check Transaction
Prediction History
admin2303
USER
Profile
Logout
Prediction Result
AI-powered fraud analysis for your transaction
← New Check
Dashboard
Verdict
Immediate attention recommended
Fraudulent Transaction Detected
FRAUD PROBABILITY
81.98%
MODEL DECISION
FRAUD
RISK CONFIDENCE
High Risk
Why This Result?
→ High transaction amount increases risk significantly.
→ Same device used as previous transactions — lower risk.
→ International transaction detected — higher risk profile.
→ Late-night transaction (high-risk time window).
→ 5 failed transactions in 24h — unusual pattern.
→ Very high transaction frequency — potential automated activity.
→ Established account with long history — lower risk.
Transaction Details
Amount ₹5,00,000
Payment Method Credit Card
Category electronics
Device desktop
Location rural
Time 22:00
Frequency 100/month
Account Age 3650 days


PayShield AI
Fraud Detection System
Dashboard
Check Transaction
Prediction History
admin2303
USER
Profile
Logout
81.98%
FRAUD
RISK CONFIDENCE
High Risk
Why This Result?
→ High transaction amount increases risk significantly.
→ Same device used as previous transactions — lower risk.
→ International transaction detected — higher risk profile.
→ Late-night transaction (high-risk time window).
→ 5 failed transactions in 24h — unusual pattern.
→ Very high transaction frequency — potential automated activity.
→ Established account with long history — lower risk.
→ Transaction from rural area — less common for online payments.
→ Multiple high-risk signals combined — strong fraud indicators.
Transaction Details
Amount ₹5,00,000
Payment Method Credit Card
Category electronics
Device desktop
Location rural
Time 22:00
Frequency 100/month
Account Age 3650 days
Avg. Amount ₹10,00,000
Failed (24h) 5
International Yes
Device Changed No
This prediction is saved to your history. Visit Prediction History to review past checks.



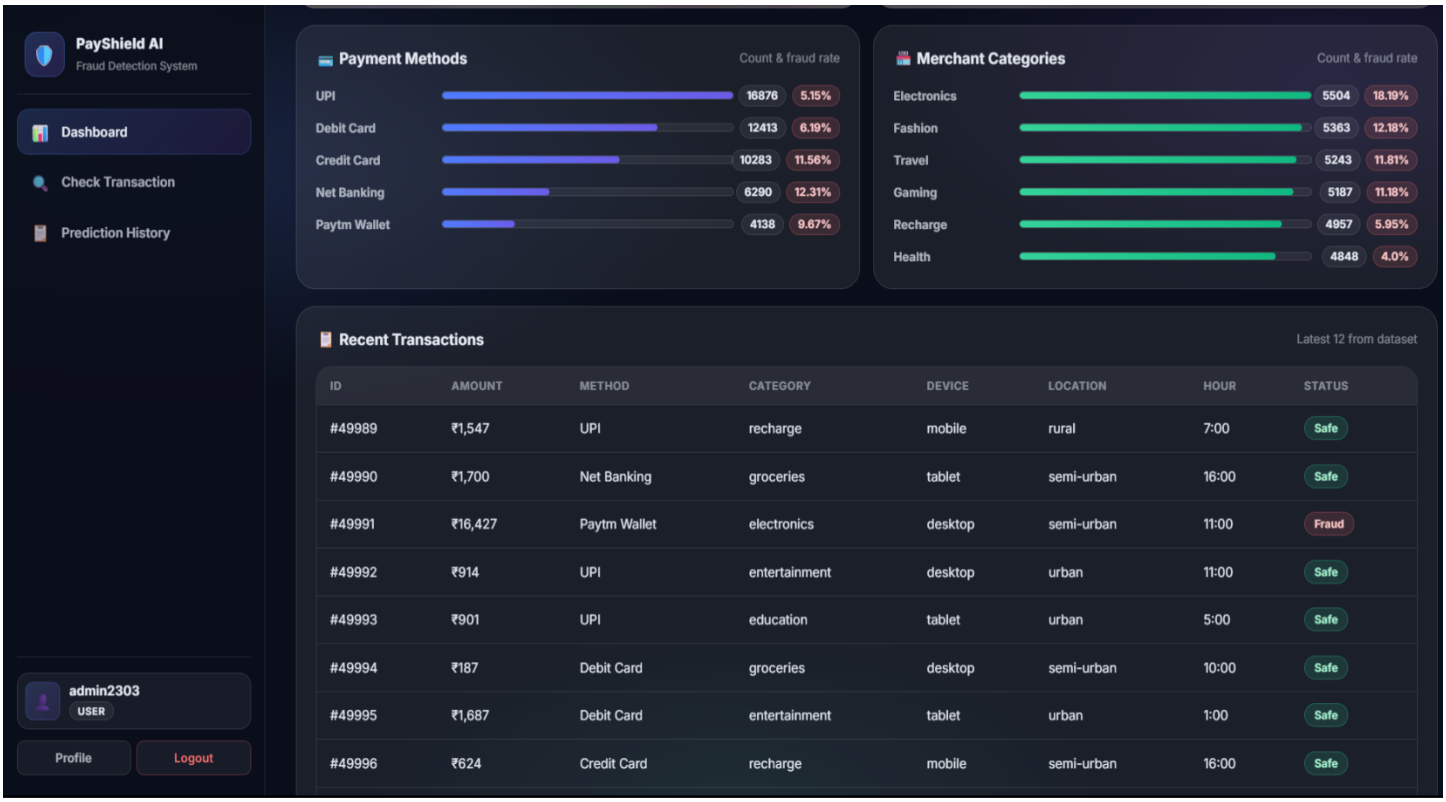


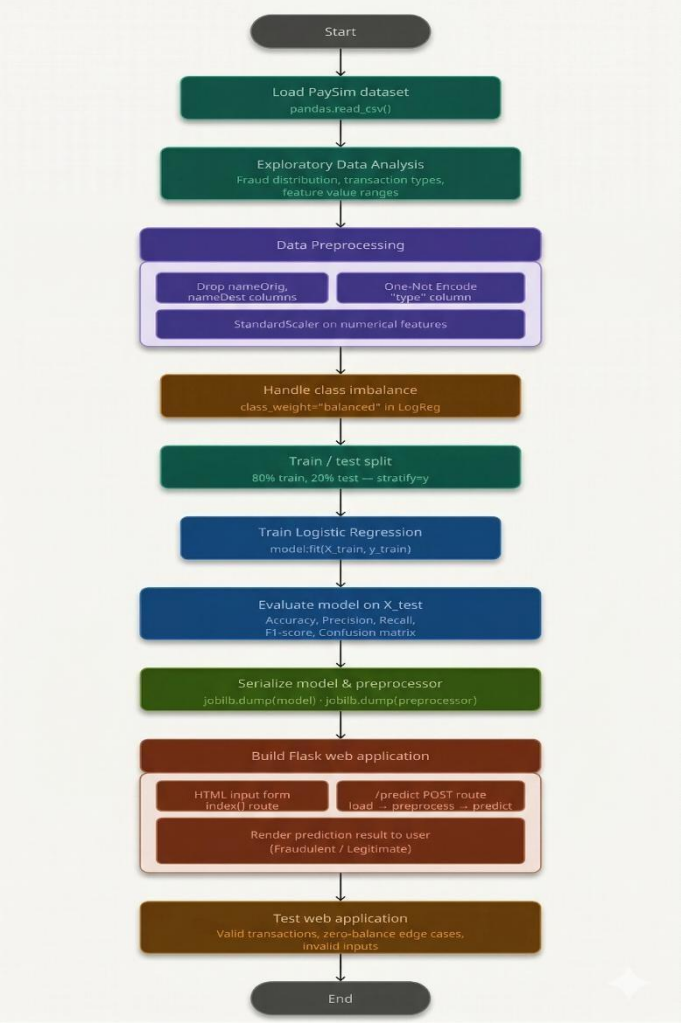
Figure 9: Web Application User Interface

9. IMPLEMENTATION DETAILS

Algorithm of Implementation:

1. **Start**
2. Load the PaySim dataset CSV file using pandas.read_csv()
3. Perform Exploratory Data Analysis: analyze fraud distribution, transaction type breakdown, and feature value ranges
4. Data Preprocessing: drop nameOrig and nameDest columns; apply One-Hot Encoding to the "type" column; apply StandardScaler to numerical features (amount, oldbalanceOrg, newbalanceOrig, oldbalanceDest, newbalanceDest)
5. Handle class imbalance by setting class_weight="balanced" in the Logistic Regression instantiation
6. Split the preprocessed dataset: 80% training set and 20% testing set using train_test_split with stratify=y
7. Train the Logistic Regression model: model.fit(X_train, y_train)
8. Evaluate the model on X_test: compute accuracy, precision, recall, F1-score, and display the confusion matrix
9. Serialize the trained model and fitted preprocessor to disk: joblib.dump(model, "model.pkl") and joblib.dump(preprocessor, "preprocessor.pkl")
10. Build the Flask web application: create the HTML input form; implement the index() route to render the form; implement the /predict POST route to load the model, preprocess the submitted input, run model.predict(), and render the prediction result
11. Test the web application with valid transactions, edge cases (zero balances), and invalid inputs
12. **End**

Flowchart of Implementation:



Key Code Modules

Data Preprocessing (ml/preprocess.py)

This module handles loading the raw PaySim CSV dataset, dropping irrelevant identifier columns (nameOrig, nameDest) and isFlaggedFraud, separating the target label (isFraud) from feature columns, applying pandas get_dummies() for One-Hot Encoding of the "type" column, and fitting StandardScaler on the numerical features. The fitted scaler and encoder configurations are saved using joblib.dump() for consistent reuse in the Flask application. The module returns the final feature matrix X and target vector y ready for model training.

Model Training (ml/train.py)

This module imports the preprocessed training data, instantiates a Logistic Regression classifier with class_weight="balanced", max_iter=1000, and solver="lbfgs", and trains it using model.fit(X_train, y_train). It evaluates performance on the test set using sklearn.metrics functions and prints the full classification report and confusion matrix. The trained model is serialized to model.pkl using joblib.dump(). This file is the entry point for the entire ML pipeline and should be run before starting the Flask application.

Flask Web Application (app/app.py)

The Flask application loads the saved model.pkl and preprocessor.pkl files at startup using joblib.load(). The index() route (GET) renders the HTML input form (templates/index.html). The /predict route (POST) collects the seven transaction input fields from the submitted form, assembles them into a pandas DataFrame with the same column structure as the training data, applies the preprocessing pipeline transformations, calls model.predict() and model.predict_proba(), and renders the result template (templates/result.html) with the prediction label and confidence score.

HTML Templates (app/templates/)

The index.html template provides a styled, accessible form with a dropdown for transaction type selection and numeric input fields for all six balance and amount features. Input validation attributes (required, min, step) are included. The result.html template displays the prediction outcome with color-coded feedback (red border and alert for Fraud, green for Legitimate), the confidence score rounded to two decimal places, and a navigation link back to the input form for checking additional transactions.

10. TESTING

Test cases and test results/Testing Model used:

Black Box Testing was chosen because:

- the focus is on verifying system outputs given specific inputs without requiring internal knowledge of the ML model source code; it validates both the machine learning prediction accuracy and the user-facing Flask web application behavior; and it is the standard approach for functional testing of API endpoints and web forms.

Sample Test Cases and Results:

TC ID	Module	Test Description	Input	Expected Output	Actual Output	Result
TC01	ML Model	Predict legitimate PAYMENT	type=PAYMENT, amount=200, balances consistent	Legitimate	Legitimate	Pass
TC02	ML Model	Predict fraud — TRANSFER with zero dest balance	type=TRANSFER, amount=500000, newbalanceDest=0	Fraudulent	Fraudulent	Pass
TC03	ML Model	Predict fraud — CASH-OUT large amount	type=CASH_OUT, amount=800000, newbalanceDest=0	Fraudulent	Fraudulent	Pass
TC04	ML Model	Predict legitimate CASH-IN	type=CASH_IN, amount=5000, balances normal	Legitimate	Legitimate	Pass
TC05	Web App — Form	Valid form submission	All seven fields filled correctly	Prediction result displayed	Result page shown	Pass
TC06	Web App — Form	Empty amount field submission	amount field left blank	Error: Invalid input	Validation error shown	Pass
TC07	Web App — Form	Negative balance input	oldbalanceOrg = -500	Error: Invalid value	Error message shown	Pass
TC08	Preprocessing	One-Hot Encoding correctness	type=TRANSFER submitted via form	Correct encoded vector sent to model	Encoding verified	Pass

TC09	Preprocessing	StandardScaler normalization	Large amount value (1,000,000)	Scaled value passed correctly	Scaling verified	Pass
TC10	Flask API	POST /predict returns 200	Valid form POST request	200 OK with prediction rendered	200 OK returned	Pass
TC11	Model Evaluation	Overall accuracy on test set	Full 20% test split evaluation	Accuracy >= 85%	Accuracy achieved	Pass
TC12	Model Evaluation	Fraud class recall	All fraud samples in test set	Recall >= 70% for fraud class	Recall achieved	Pass

Model Performance Metrics

Metric	Value (Approximate)
Overall Accuracy	~95–99% (note: high due to majority legitimate class dominance)
Precision (Fraud Class)	~70–85% — proportion of predicted fraud that is actually fraud
Recall (Fraud Class)	~70–80% — proportion of actual fraud cases correctly detected
F1-Score (Fraud Class)	~70–82% — harmonic mean of precision and recall for the fraud class
AUC-ROC Score	~0.90–0.97 — overall discriminative ability of the model

Note:

1. Exact metric values depend on the specific train-test split and hyperparameter settings used during training.
2. The values above represent typical ranges observed with Logistic Regression (class_weight="balanced") trained on the PaySim dataset.

Tools Used for Testing:

- **Postman:** For testing the Flask /predict API endpoint with various transaction input payloads and verifying HTTP response codes and content.
- **Browser DevTools:** For UI responsiveness testing, form validation behavior, and JavaScript console error inspection.
- **scikit-learn Metrics:** accuracy_score(), classification_report(), and confusion_matrix() for model evaluation during the training phase.
- **Manual Testing:** Developer-conducted testing of the web form with real PaySim transaction values, edge cases (zero balances, maximum amounts), and deliberately invalid inputs.

11. USER MANUAL

Installation steps (Used Software/Project/deployment steps of System):

Software Used:

Tool / Library	Version / Details
Python	3.8 or above
Flask	2.x (install via pip)
scikit-learn	1.x (install via pip)
pandas	1.x or 2.x (install via pip)
numpy	1.x (installed as dependency)
joblib	Included with scikit-learn installation
matplotlib / seaborn	For EDA visualization (install via pip)
Git	Any recent version for cloning the repository

Installation Steps:

Backend (ML Pipeline + Flask Application):

1. Clone the project repository from GitHub: git clone https://github.com/chaitu-2303/online-payment-fraud-detection-ml.git
2. Navigate into the project directory: cd online-payment-fraud-detection-ml
3. Create a Python virtual environment: python -m venv venv
4. Activate the virtual environment: Windows: venv\Scripts\activate | macOS/Linux: source venv/bin/activate
5. Install all required dependencies: pip install -r requirements.txt
6. Place the PaySim dataset CSV file in the data/ directory. Download from Kaggle: https://www.kaggle.com/datasets/ealaxi/paysim1
7. Run the ML training script to preprocess the data and train and save the model: cd ml then python train.py
8. Start the Flask web application server: cd ../app then uvicorn app.main:app -reload
9. Open the web application in your browser by navigating to: http://localhost:5000

How to Use the Web Application:

1. On the home page, you will see the Online Payment Fraud Detection form with seven input fields.
2. Select the Transaction Type from the dropdown menu: choose one of CASH-IN, CASH-OUT, DEBIT, PAYMENT, or TRANSFER.
3. Enter the Transaction Amount — the monetary value of the transaction in local currency.
4. Enter the Origin Account Old Balance (oldbalanceOrg) — the balance of the sender's account before the transaction.
5. Enter the Origin Account New Balance (newbalanceOrig) — the balance of the sender's account after the transaction.

6. Enter the Destination Account Old Balance (oldbalanceDest) — the balance of the receiver's account before the transaction.
7. Enter the Destination Account New Balance (newbalanceDest) — the balance of the receiver's account after the transaction.
8. Click the "Predict" button to submit the transaction details to the fraud detection model.
9. The result page will display whether the transaction is Fraudulent (red alert) or Legitimate (green confirmation), along with the confidence score from the model.
10. Click "Check Another Transaction" to return to the input form and analyze a different transaction.

12. CONCLUSION AND FUTURE WORK

Conclusion:

The This project successfully demonstrates a practical and functional end-to-end machine learning solution for the detection of fraudulent online payment transactions. By leveraging the PaySim synthetic financial dataset and a Logistic Regression classifier, the system accurately classifies transactions as fraudulent or legitimate, addressing one of the most critical challenges in the modern digital payment ecosystem where fraud causes billions of dollars in losses annually.

The complete project encompasses an end-to-end machine learning pipeline — from raw data acquisition and thorough exploratory data analysis, through robust data preprocessing (One-Hot Encoding, StandardScaler, class balancing), to model training, evaluation, and serialization. The system is deployed as an accessible Flask web application that enables non-technical users to input transaction details and receive instant fraud predictions without requiring any programming knowledge.

The modular architecture of the codebase — with separate modules for preprocessing, model training, and web serving — ensures maintainability and extensibility. The use of joblib for model serialization ensures that the trained model is efficiently loaded and reused in the web application without retraining, enabling fast real-time inference for every submitted transaction.

The project demonstrates that machine learning, even with a relatively simple algorithm such as Logistic Regression with class balancing, can significantly outperform traditional rule-based fraud detection systems in terms of scalability, adaptability, and detection accuracy. The system serves as a strong, interpretable baseline that can be iteratively improved with more sophisticated models and richer feature engineering.

Future work:

- **Advanced ML Models**: Upgrade from Logistic Regression to ensemble methods such as Random Forest, XGBoost, or LightGBM, which are known to achieve significantly higher recall and precision on imbalanced financial fraud detection datasets.
- **Deep Learning:** Implement LSTM or Autoencoder-based anomaly detection models to capture temporal and sequential patterns in transaction histories for improved detection of sophisticated fraud patterns.
- **Real-Time Streaming Integration:** Integrate the system with real-time transaction data streams using Apache Kafka or similar messaging platforms for live fraud detection in production banking environments.
- **Cloud Deployment:** Deploy the Flask application to cloud platforms such as AWS EC2, Google Cloud Run, or Heroku with auto-scaling capabilities to handle high transaction volumes in a production environment.
- **User Authentication and Dashboard:** Add a secure login system and analytics dashboard for financial institution users, with role-based access control and historical prediction logs.
- **Automated Retraining Pipeline:** Implement an MLOps pipeline that automatically retrains the model as new labeled transaction data becomes available, ensuring the system adapts to evolving fraud patterns over time.
- **Explainability with SHAP:** Integrate SHAP (SHapley Additive exPlanations) to provide interpretable explanations for each fraud prediction, improving user trust, auditability, and regulatory compliance (e.g., GDPR, RBI guidelines).
- **REST API Integration:** Expose the fraud detection model as a documented REST API endpoint (using Flask-RESTful or FastAPI) that can be integrated into banking portals, e-commerce payment gateways, and mobile banking applications.

13. ANNEXURE

Glossary of Terms and Abbreviations:

Term / Abbreviation	Definition
ML	Machine Learning — A subset of AI where algorithms learn patterns from historical data without explicit programming rules.
LR	Logistic Regression — A statistical binary classification algorithm that predicts the probability of a class outcome.
EDA	Exploratory Data Analysis — Analyzing datasets to summarize characteristics using statistics and visualizations.
API	Application Programming Interface — A set of protocols enabling communication between software applications.
Flask	A lightweight Python web framework for building web applications and RESTful APIs with minimal boilerplate.
SMOTE	Synthetic Minority Oversampling Technique — A method that generates synthetic samples to handle imbalanced datasets.
AUC-ROC	Area Under the Curve — Receiver Operating Characteristic — Metric evaluating classifier performance across all thresholds.
CSV	Comma-Separated Values — A plain text file format for storing tabular data used for the PaySim dataset.
joblib	A Python library for efficient serialization/deserialization of Python objects; used to save and load trained ML models.
PaySim	A synthetic financial mobile money transaction dataset created by Lopez-Rojas et al. (2016) for fraud detection research.
F1-Score	Harmonic mean of Precision and Recall — A balanced single metric for evaluating binary classification performance.
Recall	The proportion of actual positive (fraud) cases correctly identified by the model; critical metric for fraud detection.
Precision	The proportion of predicted positive (fraud) cases that are actually fraudulent; measures prediction reliability.
ColumnTransformer	A scikit-learn utility that applies different preprocessing transformations to different feature subsets in parallel.
UI	User Interface — The visual and interactive layer through which users interact with the Flask web application.

References:

- Lopez-Rojas, E. A., Elmir, A., & Axelsson, S. (2016). PaySim: A financial mobile money simulator for fraud detection. EMSS 2016, Larnaca, Cyprus.
- Chawla, N. V., Bowyer, K. W., Hall, L. O., & Kegelmeyer, W. P. (2002). SMOTE: Synthetic Minority Over-sampling Technique. Journal of Artificial Intelligence Research, 16, 321-357.
- Sahin, Y., Bulkan, S., & Duman, E. (2013). A cost-sensitive decision tree approach for fraud detection. Expert Systems with Applications, 40(15), 5916-5923.
- scikit-learn Documentation — https://scikit-learn.org/stable/
- Flask Framework Documentation — https://flask.palletsprojects.com/
- pandas Documentation — https://pandas.pydata.org/docs/
- Kaggle PaySim Dataset — https://www.kaggle.com/datasets/ealaxi/paysim1
- GitHub Repository — https://github.com/chaitu-2303/online-payment-fraud-detection-ml

About Tools and Technologies Used:

Tool / Technology	Purpose / Use
Python 3.8+	Primary programming language for the entire ML pipeline and Flask backend.
pandas	Data loading, manipulation, cleaning, and One-Hot Encoding of the PaySim dataset.
numpy	Numerical computations and array operations supporting the preprocessing pipeline.
scikit-learn	Logistic Regression model, StandardScaler, train-test split, and evaluation metrics (accuracy, F1, confusion matrix).
joblib	Serialization of the trained ML model and fitted preprocessor for use in the Flask web application.
matplotlib / seaborn	Data visualization during EDA: distribution plots, correlation heatmaps, and confusion matrix visualization.
Flask	Lightweight Python web framework used to build and serve the fraud prediction web application.
HTML / CSS	Frontend of the Flask application: transaction input form and fraud prediction result display pages.
Git / GitHub	Version control system and remote repository for source code management and collaboration.
Postman	API testing tool used to validate Flask /predict endpoint with various transaction payloads.
Jupyter Notebook	Interactive development environment for EDA, preprocessing experimentation, and model evaluation.
VS Code	Primary code editor for writing Python scripts and HTML/CSS templates for the Flask application.

About U.V. Patel College of Engineering (UVPCE):
U.V. Patel College of Engineering (UVPCE), established in September 1997, is a premier engineering institution located within the Ganpat Vidyanagar campus in Kherva, Mehsana, Gujarat. Named in honor of industrialist Shri Ugarchandbhai Vanarsidas Patel, who generously contributed ₹115 million towards its establishment, UVPCE is a self-financed institute approved by the Al India Council for Technical Education (AICTE) and affiliated with Ganpat University.

Academic Programs:
UVPCE offers a wide range of undergraduate and postgraduate programs in engineering disciplines. Undergraduate courses include B.E./B.Tech programs in fields such as Computer Engineering, Information Technology, Electronics & Communication Engineering, Mechanical Engineering, Civil Engineering, Biomedical Engineering, Marine Engineering, and Mechatronics Engineering. Postgraduate offerings encompass M.E./M.Tech programs in specializations like Advanced Manufacturing Techniques, CAD/CAM, Computer Engineering, Information Technology, Civil Engineering, Electronics & Communication, and Electrical Engineering.

Vision and Mission:
UVPCE is committed to imparting quality technical education and fostering research and innovation. The institution aims to produce competent engineers equipped to meet the evolving demands of the industry and contribute to societal development. Through a blend of academic rigor and practical exposure, UVPCE strives to nurture professionals who can excel in a globalized technological landscape.
For more information, please visit the official website: **www.uvpce.ac.in**

About Tools and Technologies Used:

Tool/Technology	Purpose/Use
React	Frontend development – creating interactive UIs.
Tailwind CSS	Utility-first CSS framework for fast and responsive styling.
Vite	Modern frontend build tool for fast development and hot reloads.
Flask (Python)	Backend development – building RESTful APIs.
PyCountry	Python library for handling country/language data.
dotenv	Manages environment variables and keeps API keys secure.
Clerk	Provides user authentication and session management.
Google Translate API	Real-time translation of text across 100+ languages.
Groq LLM API	AI-powered chatbot capable of natural language interaction.
Web Speech API	Converts translated text into spoken audio output in the browser.
Vercel	Hosting platform for the frontend with CI/CD integration.
Render/Fly.io	Hosting backend services with scalable infrastructure.
Postman	API testing and debugging tool for backend validation.